WORKSHOP BUILD GUIDE

Build Your AI Project Manager

Take-home reference: the concepts, every code step, and how to run it — CrewAI + local Ollama.

This is your companion for the hands-on session and your reference afterward. The runnable files (pm_crew.py, virtual_office.py) are in the code bundle; this guide explains what each step does and why.

The mental model

- **Agent —** an LLM plus tools, memory, and a loop. Strip the hype and that's all it is.
- **ReAct loop —** each turn the agent Thinks, takes an Action (calls a tool), Observes the result, and repeats until the goal is met. CrewAI runs this loop for you.
- **Crew —** one agent for a focused job; a crew of specialists for complex work. Sequential = a pipeline; hierarchical = a manager delegates.
- **Local stack —** CrewAI orchestrates, LiteLLM speaks to any model, Ollama runs open weights (Llama 3.1 / Qwen 2.5) on your machine. No API keys, no data leaving the laptop.

Setup

Install a local model once, create a clean Python environment, and add the framework.

```
# 1. install Ollama (ollama.com), then pull open models
ollama pull llama3.1          # ~4.7 GB, runs fully local
ollama pull qwen2.5           # a strong alternative

# 2. a clean Python environment + the framework
python -m venv .venv && source .venv/bin/activate
pip install crewai crewai-tools pydantic

# 3. sanity check: does the local model answer?
ollama run llama3.1 "reply with one word: ready"
```

Build the PM assistant, step by step

1. Your first agent

An Agent, a Task, and a Crew to run it — pointed at the local Ollama model.

```
from crewai import Agent, Task, Crew, Process, LLM


# point CrewAI at the LOCAL model served by Ollama
llm = LLM(model="ollama/llama3.1",
          base_url="http://localhost:11434")


planner = Agent(
    role="Project Planner",
    goal="Turn a project brief into a clear task list",
    backstory="A sharp PM who makes vague briefs actionable.",
    llm=llm, verbose=True,
)


task = Task(
    description="Break down this brief into tasks:\n{brief}",
    expected_output="A numbered task list with an owner each",
    agent=planner,
)


crew = Crew(agents=[planner], tasks=[task], process=Process.sequential)
print(crew.kickoff(inputs={"brief": open("brief.md").read()}))
```

Why: One agent + one task + a crew is a working assistant in about 20 lines.

2. Give it a tool

Tools are plain Python functions. If the model decides it needs one, CrewAI calls it and feeds the result back.

```
from crewai.tools import tool
import json


@tool("save_tasks")
def save_tasks(tasks_json: str) -> str:
    "Save a JSON task list to project_state.json."
```

```
    data = json.loads(tasks_json)
    with open("project_state.json", "w") as f:
        json.dump(data, f, indent=2)
    return f"saved {len(data)} tasks"


# the agent may now decide to call this tool
planner.tools = [save_tasks]
```

Why: Expose only the tools an agent truly needs (least privilege).

3. Typed output, not prose

Make the planner return a validated Pydantic object so the next step can trust the data.

```
from pydantic import BaseModel
from typing import List


class TaskItem(BaseModel):
    title: str
    owner: str
    estimate_days: int


class Plan(BaseModel):
    tasks: List[TaskItem]


plan_task = Task(
    description="Break the brief into tasks:\n{brief}",
    expected_output="A Plan object",
    agent=planner,
    output_pydantic=Plan,     # validated, typed output
)
```

Why: Typed output is what makes agents composable — no regex-parsing of prose.

4. Add two specialists

A risk analyst and a status writer, each with a single, narrow responsibility.

```
risk = Agent(
    role="Risk Analyst",
    goal="Flag the top risks, with one mitigation each",
    backstory="A cautious PM who has watched projects slip.",
    llm=llm)

writer = Agent(
    role="Status Writer",
    goal="Draft a short status update for stakeholders",
    backstory="Writes clear, calm updates.",
    llm=llm)

risk_task = Task(description="Review the plan. List the top risks.",
                 expected_output="A risk register", agent=risk)

status_task = Task(description="Write a status update from the plan.",
                   expected_output="A 5-line update", agent=writer)
```

Why: Pass context between tasks with context=[...] so each agent sees the previous output.

5. Run the PM crew

Wire the three agents into one sequential crew and run it on a real brief.

```
crew = Crew(
    agents=[planner, risk, writer],
    tasks=[plan_task, risk_task, status_task],
    process=Process.sequential,     # planner -> risk -> writer
    verbose=True,
)

result = crew.kickoff(inputs={"brief": open("brief.md").read()})
print(result)      # tasks + risk register + a status update
```

Why: That's your AI project manager — in about 40 lines.

Intermezzo: the virtual office (NexaPaw)

Switch from a fixed pipeline to a named Lead that delegates to a specialist team — the pattern behind Felix's NexaPaw office, where every Claude Code role is an agent (a CEO and CTO above project leads above specialist engineers), visualized live in a 3D HQ. You define the staff; the Lead decides who does what, and when.

```
# a NexaPaw-style project team: a Lead delegates to specialists
lead = Agent(role="Project Lead",
             goal="Deliver the feature; assign work to the right role",
             backstory="Runs a NexaPaw project team.", llm=llm)

cv  = Agent(role="CV Engineer", goal="Build + evaluate the model", llm=llm)
be  = Agent(role="Backend Engineer", goal="APIs and data", llm=llm)
fe  = Agent(role="Frontend Engineer", goal="The user interface", llm=llm)
qa  = Agent(role="QA Engineer", goal="Test against the spec", llm=llm)

office = Crew(
    agents=[cv, be, fe, qa],
    tasks=[deliver_feature],        # one high-level goal
    process=Process.hierarchical,   # the Lead delegates
    manager_agent=lead,             # a named manager, not just an LLM
    verbose=True,
)
office.kickoff(inputs={"brief": "Ship cross-camera re-ID v2"})
```

HONEST TRADE-OFF

A virtual office is powerful and harder to control. More agents = more model calls = more time and nondeterminism. Reach for it only when a single agent plateaus.

Make it real

Validate outputs, cap iterations, and keep consequential actions behind a person.

```
from pathlib import Path
```

```
# gate risky actions behind a human — draft, never send
@tool("send_email")
def send_email(to: str, body: str) -> str:
    "Save a draft for human approval; does NOT send."
    Path("outbox.md").write_text(body)
    return "DRAFT saved to outbox.md — needs approval"


# stop a stuck agent from looping forever
planner = Agent(role="Project Planner", goal="...",
                llm=llm, max_iter=8, max_retry_limit=2)
```

- Turn on verbose / logging to see each thought, tool call, and result.
- Trace tokens and latency — local models are free but slower; cache repeated calls.
- Evaluate on a fixed set of briefs with known-good outputs before you trust it.
- Local-first keeps customer data off third-party servers (helps with PDP Law).
- Watch for prompt injection when agents read external text, email, or web pages.

SIMPLE RULE

An agent should propose; a person should dispose — for anything that matters (send, pay, delete, or decisions about people).

Run the finished code

- ollama pull llama3.1 (once)
- pip install -r requirements.txt
- python pm_crew.py --brief brief.md — plans tasks, flags risks, drafts a status update
- python virtual_office.py — the hierarchical virtual office
- If an import or argument fails, check your installed CrewAI version against its current docs.

Prepared by Felix Yustian Setiono for the MurzEdu Program (ecommurz). Code is a starting point — adapt it to your stack.